Python: pensar, probar, construir

Primeros pasos en un lenguaje de programación

```
nombre: str =
"Lámpara"
unidades: int = 12

if unidades > 0:
    print("Producir")
```

CLASE 2

La primera clase dejó el marco; hoy aparece el código

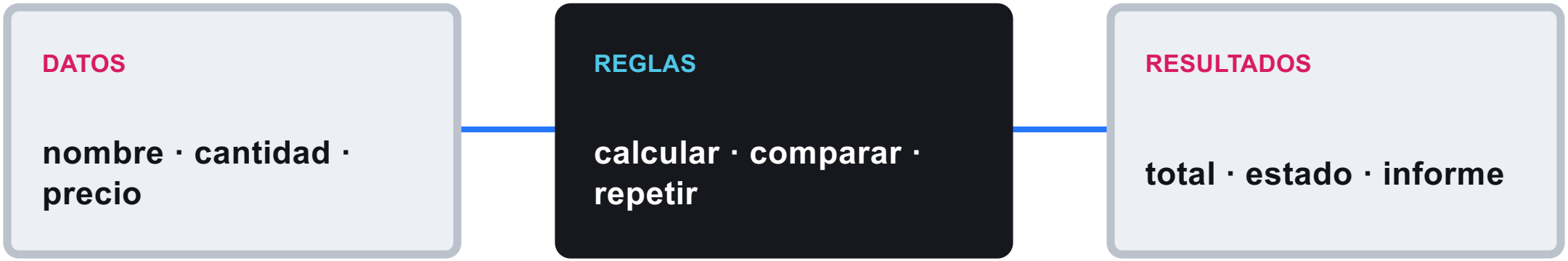
5 MIN

Todavía no construimos ningún programa.

- Vimos para qué vamos a programar.
- Reconocimos herramientas y entornos de trabajo.

¿Qué diferencia hay entre describir una solución y escribirla para que una computadora la ejecute?

Todo programa transforma datos mediante reglas



Hoy vamos a construir las reglas con Python.

Python prioriza la legibilidad y la prueba rápida

Es un lenguaje de propósito general, interpretado y de tipado dinámico.

Python es un lenguaje de programación utilizado para crear aplicaciones, automatizar tareas, analizar datos y desarrollar sistemas de inteligencia artificial.

Se caracteriza por:
- Una sintaxis clara y legible.
- Poder ejecutarse en distintos sistemas.
- Contar con una gran cantidad de bibliotecas.
- Permitir desde programas simples hasta proyectos complejos.

PRIMERA PRUEBA

```
mensaje: str = "Hola, Python"
print(mensaje)
```

Documentación, visualización y práctica

Tutorial oficial	docs.python.org/es/3/tutorial/
Jupyter en navegador	https://jupyter.org/
Ejecución paso a paso	pythontutor.com/python-compiler.html
Ejercicios	exercism.org/tracks/python

La documentación se consulta; la sintaxis no se memoriza completa.

Vamos a usar Jupyter, que corre en el navegador

ABRIR

jupyter.org

- Esperar a que el kernel esté listo.
- Crear un notebook con Python.
- Subir los archivos .ipynb desde el panel lateral.
- Descargar una copia al terminar.



Un notebook alterna explicación, código y resultado

MARKDOWN

Título, consigna o explicación

CÓDIGO

```
Instrucciones que Python ejecuta
```

SALIDA

El resultado o el error obtenido

En un notebook importa el orden

CELDA 1

```
precio: float = 18500.0
unidades: int = 12
```

CELDA 2

```
total: float = precio * unidades
print(total)
```

Si ejecutamos la segunda celda antes que la primera, Python todavía no conoce precio ni unidades.

Regla práctica

Run All antes de entregar o mostrar un notebook.

Variables: nombres y valores

EJEMPLO

```
nombre: str = "Lámpara modular"
unidades: int = 12
precio: float = 18500.0
disponible: bool = True
```

nombre

qué representa

valor

el dato concreto

Los nombres claros reducen la cantidad de código que tenemos que descifrar.

Anotamos tipos para hacer visible nuestra intención

TIPOS BÁSICOS

```
unidades: int = 12
precio: float = 18500.0
nombre: str = "Lámpara"
disponible: bool = True
```

Python no exige estas anotaciones para ejecutar.

- Documentan qué dato esperamos.
- Ayudan a detectar inconsistencias con editores y analizadores.
- Facilitan el trabajo colaborativo y la asistencia de IA.
- No convierten a Python en un lenguaje de tipado estático.

PRÁCTICA 1

Expandir la representación

CONSIGNA

Copiá el ejemplo de la lámpara y agregá dos variables relevantes con tipos diferentes:

```
nombre: str = "Lámpara modular"
unidades: int = 12
precio: float = 18500.0
disponible: bool = True

print(type(nombre))
print(type(unidades))
print(type(precio))
print(type(disponible))
```

EXPANSIÓN

Agregá un dato que todavía pueda no tener valor o verificá los tipos con type().

COMPROBAR

Explicar por qué eligieron cada tipo.

10 MIN

input() produce texto: convertir también es una decisión

ENTRADA Y CONVERSIÓN

```
nombre: str = input("Producto: " )
unidades: int = int(input("Unidades: "))
precio: float = float(input("Precio: "))

print(nombre, unidades, precio)
```

- **input():** recibe texto.
- **int():** convierte a entero.
- **float():** convierte a decimal.
- **print():** muestra un resultado.

¿Qué pasaría si escribimos doce en lugar de 12?

Las expresiones producen resultados

CÁLCULO

```
subtotal: float = unidades * precio
desperdicio: float = 0.08
total: float = subtotal * (1 + desperdicio)

mensaje: str = f"Total: ${total:,.2f}"
print(mensaje)
```

Operadores que usamos hoy

`+` `-` `*` `/` `**` `%`

Una f-string incorpora valores dentro de una salida legible.

PRÁCTICA 2

Agregar una operación

CONSIGNA

Incorporá un descuento y mostrale al usuario el subtotal, el descuento y el total final.

EXPANSIÓN

Permití que el usuario ingrese el porcentaje de descuento y mostrale cómo cambia el total final.

COMPROBAR

Usar dos casos de prueba y comparar.

Agregar una operación - Resolución

```
producto: str = "Lámpara"
precio_unitario: float = 25000.0
cantidad: int = 3
porcentaje_descuento: float = 10.0

subtotal: float = precio_unitario * cantidad
descuento: float = subtotal * porcentaje_descuento / 100
total_final: float = subtotal - descuento

print(f"Producto: {producto}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"Descuento ({porcentaje_descuento}%): ${descuento:.2f}")
print(f"Total final: ${total_final:.2f}")
```

Agregar una operación – Resolución + Expansión

```
producto: str = "Lámpara"
precio_unitario: float = 25000.0
cantidad: int = 3

porcentaje_descuento: float = float(
    input("Ingresá el porcentaje de descuento: ")
)

subtotal: float = precio_unitario * cantidad
descuento: float = subtotal * porcentaje_descuento / 100
total_final: float = subtotal - descuento

print(f"\nProducto: {producto}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"Descuento ({porcentaje_descuento}%): ${descuento:.2f}")
print(f"Total final: ${total_final:.2f}")
```

Una condición cambia qué instrucciones se ejecutan

IF · ELIF · ELSE

```
producto: str = "Lámpara"
precio_unitario: float = 25000.0
cantidad: int = 3
porcentaje_descuento: float = 10.0
presupuesto: float = 65000.0

subtotal: float = precio_unitario * cantidad
descuento: float = subtotal * porcentaje_descuento / 100
total: float = subtotal - descuento

if total <= presupuesto:
    estado: str = "Dentro del presupuesto"
elif total <= presupuesto * 1.10:
    estado = "Requiere un ajuste menor"
else:
    estado = "Supera el presupuesto"

print(f"Producto: {producto}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"Descuento: ${descuento:.2f}")
print(f"Total final: ${total:.2f}")
print(f"Presupuesto disponible: ${presupuesto:.2f}")
print(f"Estado: {estado}")
```

- La comparación produce True o False.
- Python ejecuta una sola rama.
- La indentación define qué líneas pertenecen a cada rama.

Decisiones combinadas

DOS CRITERIOS

```
presupuesto_ok: bool = total <= presupuesto
tiempo_ok: bool = horas_necesarias <= horas_disponibles


if presupuesto_ok and tiempo_ok:
    print("Producción viable")
```

and se cumplen ambas condiciones

or se cumple al menos una

not invierte un valor booleano

PRÁCTICA 3

Ampliar la regla

CONSIGNA

Agregá horas_disponibles y decidí si la producción es viable por presupuesto y por tiempo.

EXPANSIÓN

Conservá mensajes específicos para saber cuál de los dos criterios falla.

COMPROBAR

Probar cuatro combinaciones de True y False.

10 MIN

Ampliar la regla – Resultado + expansión

```
producto: str = "Lámpara"
precio_unitario: float = 25000.0
cantidad: int = 3
porcentaje_descuento: float = 10.0

presupuesto: float = 65000.0
horas_necesarias: float = 8.0
horas_disponibles: float = 10.0

subtotal: float = precio_unitario * cantidad
descuento: float = subtotal * porcentaje_descuento / 100
total: float = subtotal - descuento

presupuesto_ok: bool = total <= presupuesto
tiempo_ok: bool = horas_necesarias <= horas_disponibles

if presupuesto_ok and tiempo_ok:
    resultado: str = "Producción viable"
elif not presupuesto_ok and tiempo_ok:
    resultado = "Producción no viable: supera el presupuesto"
elif presupuesto_ok and not tiempo_ok:
    resultado = "Producción no viable: no hay suficiente tiempo"
else:
    resultado = "Producción no viable: supera el presupuesto y no hay suficiente tiempo"

print(f"Producto: {producto}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"Descuento: ${descuento:.2f}")
print(f"Total final: ${total:.2f}")
print(f"Presupuesto disponible: ${presupuesto:.2f}")
print(f"Horas necesarias: {horas_necesarias}")
print(f"Horas disponibles: {horas_disponibles}")
print(f"Resultado: {resultado}")
```

Una lista reúne varios valores que necesitamos recorrer

LISTA

```
costos: list[float] = [
    45000.0,
    19200.0,
    8500.0,
]

print(costos[0])
print(len(costos))
```

Índice 0

primer elemento

len()

cantidad de elementos

La lista puede crecer:
costos.append(12000.0)

for repite una acción para cada elemento

RECORRIDO Y ACUMULACIÓN

```
total: float = 0.0

for costo in costos:
    total += costo
    print(f"Procesado: ${costo:,.2f}")

print(f"Total: ${total:,.2f}")
```

- costo toma un valor distinto en cada vuelta.
- total existe antes de comenzar el recorrido.
- El bloque indentado se repite.

Antes de ejecutar: ¿qué valor tendrá total después de la segunda vuelta?

while repite mientras una condición sea True

REPETICIÓN CON CONDICIÓN

```
pendientes: int = 3

while pendientes > 0:
    print(f"Quedan {pendientes}")
    pendientes -= 1

print("Proceso terminado")
```

Usar while cuando no sabemos de antemano cuántas repeticiones habrá.

La condición debe poder cambiar. Si no, el bucle puede no terminar.

PRÁCTICA 4

Obtener información de una lista

CONSIGNA

Calculá el promedio de los costos recorriendo la lista.

EXPANSIÓN

Encontrá el costo mayor con un recorrido propio y compará después con max().

COMPROBAR

Probar también una lista vacía.

10 MIN

Obtener información de una lista – Resultado + Expansión

```
costos: list[float] = [
    45000.0,
    19200.0,
    8500.0,
]

print(costos[0])
print(len(costos))

total: float = 0.0

for costo in costos:
    total += costo
    print(f"Procesado: ${costo:,.2f}")

print(f"Total: ${total:,.2f}")

# Calcular el promedio
promedio: float = total / len(costos)

print(f"Promedio: ${promedio:,.2f}")

# Encontrar el costo mayor recorriendo la lista
costo_mayor: float = costos[0]

for costo in costos:
    if costo > costo_mayor:
        costo_mayor = costo

print(f"Costo mayor con recorrido: ${costo_mayor:,.2f}")

# Comparar con la función max()
print(f"Costo mayor con max(): ${max(costos):,.2f}")
```

Una función le da nombre a una operación reutilizable

PARÁMETRO · RETORNO

```
costos: list[float] = [
    45000.0,
    19200.0,
    8500.0,
]

def calcular_total(costos: list[float]) -> float:
    total: float = 0.0

    for costo in costos:
        total += costo

    return total

total_produccion: float = calcular_total(costos)

print(f"Total de la producción: ${total_produccion:,.2f}")
```

- Recibe datos mediante parámetros.
- Encapsula una tarea concreta.
- Devuelve un resultado con return.
- Puede probarse de manera independiente.

Separar funciones permite cambiar una regla sin reescribir todo

DOS RESPONSABILIDADES

```
def aplicar_descuento(subtotal: float, porcentaje: float) -> float:
    descuento: float = subtotal * porcentaje / 100
    return subtotal - descuento


def entra_en_presupuesto(total: float, presupuesto: float) -> bool:
    return total <= presupuesto


subtotal: float = 75000.0
porcentaje_descuento: float = 10.0
presupuesto: float = 70000.0

total_final: float = aplicar_descuento(
    subtotal,
    porcentaje_descuento
)

presupuesto_ok: bool = entra_en_presupuesto(
    total_final,
    presupuesto
)

print(f"Subtotal: ${subtotal:,.2f}")
print(f"Total final: ${total_final:,.2f}")
print(f"¿Entra en el presupuesto?: {presupuesto_ok}")
```

Expansión: escribir una tercera función que produzca el mensaje final.

Cada función debería responder una pregunta clara.

Clases: datos y métodos

Creamos la clase:

```
# "class" indica que vamos a crear una clase.
# "Producto" es el nombre de la clase.
class Producto:

    # __init__ se ejecuta automáticamente al crear un producto.
    def __init__(
        self,                   # Representa al objeto que se está
creando.
        nombre: str,            # Dato de entrada de tipo texto.
        precio_unitario: float, # Dato de entrada de tipo decimal.
        cantidad: int           # Dato de entrada de tipo entero.
    ) -> None:                  # El constructor no devuelve un resultado.

        # Guarda los datos recibidos dentro del objeto.
        self.nombre: str = nombre
        self.precio_unitario: float = precio_unitario
        self.cantidad: int = cantidad

    # Define una acción que podrá realizar cada producto.
    def calcular_subtotal(self) -> float:

        # Devuelve el precio multiplicado por la cantidad.
        return self.precio_unitario * self.cantidad
```

instancio la clase en un objeto y lo uso:

```
lampara: Producto = Producto(
    nombre="Lámpara",
    precio_unitario=25000.0,
    cantidad=3
)



# Accedemos a sus atributos
print(f"Producto: {lampara.nombre}")
print(f"Precio unitario:
${lampara.precio_unitario:,.2f}")
print(f"Cantidad:
{lampara.cantidad}")

# Ejecutamos uno de sus métodos
subtotal: float =
lampara.calcular_subtotal()

print(f"Subtotal: ${subtotal:,.2f}")
```

Tres programas completos para avanzar según el tiempo

01

Presupuesto de actividad

variables · listas · funciones · if

02

Planificador de producción

clase · materiales · costo · viabilidad

03

Pedidos y stock

dos clases · for · while · búsqueda

Cada notebook ejecuta de principio a fin y termina con propuestas de expansión.

Errores frecuentes

`NameError`

se usó un nombre que no existe todavía

`ValueError`

la conversión no pudo realizarse

`TypeError`

la operación no tiene sentido para esos tipos

`IndentationError`

el bloque no está correctamente indentado

Leer la última línea del mensaje y localizar la celda indicada.